

BÁO CÁO TIẾN ĐỘ THỰC TẬP

Người báo cáo: Lê Tuấn Đạt

Tuần báo cáo: 17/06/2026 – 26/06/2026

Chủ đề nghiên cứu: xây dựng một mô hình dự đoán liệu các đơn hàng có bị trả lại hay không? (Returned: 1, Delivered: 0)

Các công việc đã hoàn thành

Trong tuần làm việc này, toàn bộ quy trình từ Phân tích dữ liệu (EDA), Thiết kế đặc trưng (Feature Engineering), Tái cấu trúc Pipeline mô hình hóa, Tích hợp tracking với MLflow đến Triển khai Web API đã được tối ưu hóa và hoàn thiện. Các kết quả cụ thể bao gồm:

1. Phân tích khám phá dữ liệu (EDA - Exploratory Data Analysis)

1.1. Thống kê tổng quan và Gắn nhãn

Đơn vị phân tích: Cấp đơn hàng (order_id). Nhãn mục tiêu $y=1$ nếu đơn hàng có ít nhất một sản phẩm bị trả lại, ngược lại $y=0$.

Mất cân bằng lớp nghiêm trọng (Class Imbalance): Trong tập dữ liệu eligible (gồm các đơn có trạng thái delivered và returned), tỷ lệ đơn hàng bị trả lại chỉ chiếm 6.52% (36,062 đơn), trong khi số đơn hàng giữ lại chiếm tới 93.48% (516,796 đơn).

- Hệ quả: Không thể sử dụng độ chính xác (Accuracy) làm metric đánh giá chính; thay vào đó phải sử dụng AUC-ROC, PR-AUC và F1-Score, kết hợp kỹ thuật tối ưu hóa ngưỡng quyết định (Threshold Optimization).

Phân tích lý do trả hàng: Hơn 82% nguyên nhân hoàn trả xuất phát từ các vấn đề sản phẩm và vận hành: sai kích cỡ (wrong_size - 35%), sản phẩm lỗi (defective - 20.1%), không giống mô tả (wrong_description - 17.6%), và giao hàng trễ (late_delivery - 10%).

1.2. Phân tích các mối quan hệ (Insights)

Phương thức thanh toán (payment_method) là thuộc tính mạnh nhất: Đơn hàng thanh toán bằng phương thức COD có tỷ lệ trả hàng trung bình lên tới 11.33%, gần gấp đôi so với tất cả các phương thức thanh toán trước khác (chỉ dao động từ 5.7% – 5.8%). Khách hàng trả trước có cam kết nhận hàng lớn hơn rõ rệt.

Quy mô đơn hàng có tương quan thuận với rủi ro trả hàng: Đơn hàng chứa 1 sản phẩm có tỷ lệ trả hàng là 6.5%, đơn chứa 2 sản phẩm là 6.7%, và tăng lên 7.7%

đối với đơn chứa 3 sản phẩm. Đơn hàng càng nhiều sản phẩm thì xác suất có ít nhất một sản phẩm gặp lỗi/sai kích cỡ và bị trả lại càng tăng.

Các thuộc tính nhiều/yếu:

- Loại thiết bị (device_type) có tỷ lệ trả hàng đồng đều ~6.5% trên cả Desktop, Mobile, và Tablet.
- Nhân khẩu học (Giới tính, Nhóm tuổi, Khu vực) và các đặc tính đơn thuần như Tổng giá trị đơn hàng thô (order_total_value) hay Khách hàng mới/cũ (is_first_order) đều có tỷ lệ trả hàng đi ngang quanh mức 6.5%, thể hiện tính phân tách rất kém.

Đặc tính phi tuyến tính phức tạp: Hệ số tương quan tuyến tính Pearson (r) của toàn bộ các thuộc tính thô với nhãn mục tiêu đều cực kỳ nhỏ ($|r| < 0.003$). Điều này chỉ ra mối quan hệ giữa đặc trưng và hành vi trả hàng là phi tuyến tính phức tạp, đòi hỏi việc sử dụng các thuật toán dạng cây (như XGBoost, LightGBM) thay vì các mô hình tuyến tính đơn giản.

2. Quy trình trích xuất và Lựa chọn đặc trưng (Feature Engineering & Selection)

2.1. Phương pháp trích xuất dữ liệu (Feature Extraction)

Em đã tổng hợp dữ liệu từ các bảng quan hệ liên quan (order_items, products, customers, reviews, geography) lên cấp đơn hàng (order_id).

- **Nguyên tắc chống rò rỉ dữ liệu (Leakage-Free):** Tại thời điểm khách hàng bấm Checkout, không sử dụng bất kỳ thông tin nào xuất hiện sau đó (như ngày giao hàng, lý do trả hàng, ngày trả hàng). Lịch sử mua sắm và đánh giá của khách hàng bắt buộc phải được tính toán dựa trên điều kiện thời gian: các sự kiện cũ xảy ra trước ngày đặt hàng (order_date) của đơn hàng hiện tại.

2.2. Lựa chọn đặc trưng (Feature Selection)

Loại bỏ Đa cộng tuyến (Multicollinearity): Qua phân tích ma trận tương quan, thuộc tính max_unit_price và min_unit_price có độ tương quan tuyến tính $r = 0.998$ với avg_unit_price.

Loại bỏ thuộc tính yếu: Loại bỏ hoàn toàn đặc trưng device_type để giảm nhiễu cho mô hình.

Xây dựng các đặc trưng tương tác phi tuyến: Thay vì sử dụng các biến số thô độc lập, chúng tôi thiết kế các biến tỷ lệ phản ánh trực tiếp hành vi mua sắm bất thường của khách hàng.

2.3. Danh sách các nhóm đặc trưng được chọn (Final Feature Set)

Toàn bộ các đặc trưng huấn luyện được chia thành 6 nhóm chính:

Nhóm 1: Thành phần đơn hàng (Order Composition)

- `order_total_value`: Tổng giá trị tiền của đơn hàng.
- `n_products`: Số lượng dòng sản phẩm duy nhất trong đơn.
- `n_items`: Tổng số lượng sản phẩm đặt mua.
- `price_range`: Chênh lệch giữa giá sản phẩm đắt nhất và rẻ nhất trong đơn.
- `avg_unit_price`: Giá bán trung bình của một sản phẩm trong đơn.
- `total_discount`: Tổng số tiền được giảm giá của đơn hàng.

Nhóm 2: Lịch sử mua sắm của khách hàng (Customer History)

- `customer_order_number`: Số thứ tự đơn hàng lũy kế của khách hàng (biểu thị mức độ thân thiết).
- `customer_tenure_days`: Số ngày kể từ đơn hàng đầu tiên của khách hàng đến đơn hiện tại.
- `customer_recency_days`: Số ngày kể từ đơn hàng gần nhất trước đó của khách hàng.
- `customer_avg_rating`: Điểm đánh giá trung bình lịch sử khách hàng đã chấm cho các đơn trước (chỉ tính các review đã viết trước ngày đặt đơn hiện tại).
- `customer_review_count`: Số lượng review khách hàng đã gửi trong quá khứ.

Nhóm 3: Kênh đặt hàng (Channel Features)

- `is_cod` (Binary): Nhận giá trị 1 nếu khách hàng thanh toán bằng COD, ngược lại nhận 0.
- `payment_method` & `order_source`: Các thuộc tính phân loại đã qua mã hóa.

Nhóm 4: Đặc trưng thời gian (Temporal Features)

- `order_hour`, `order_dow` (Day of week), `order_month`: Khai thác tính chu kỳ mua sắm theo giờ trong ngày, ngày trong tuần, và tháng trong năm.

Nhóm 5: Mã hóa nhãn mục tiêu lũy kế (Target-Encoded Features - Leakage-Safe)

- `category_hist_return_rate`: Tỷ lệ trả hàng lũy kế lịch sử của ngành hàng.
- `region_hist_return_rate`: Tỷ lệ trả hàng lũy kế lịch sử của khu vực địa lý.
- `payment_hist_return_rate`: Tỷ lệ trả hàng lũy kế lịch sử của phương thức thanh toán.
- Cách tính: Được tính toán thông qua kỹ thuật Expanding Mean (chỉ tính trên các đơn hàng đã phát sinh trước mốc thời gian đặt đơn hiện tại) để tránh rò rỉ thông tin tương lai.

Nhóm 6: Tương tác phi tuyến (Interaction Features)

- `items_per_product`: Tỷ lệ tổng số lượng / số sản phẩm duy nhất ($n_items / n_products$). Phát hiện hành vi mua gom số lượng lớn.
- `discount_ratio`: Tỷ lệ giảm giá thực tế trên tổng giá trị đơn hàng.

- **size_variety_ratio:** Số lượng size duy nhất chia cho số sản phẩm duy nhất trong đơn hàng. Tỷ lệ này tiến gần đến 1 cho thấy khách hàng đang mua nhiều kích cỡ cho cùng một sản phẩm (hành vi mua thử size và sẽ trả lại sản phẩm không vừa).
- **color_variety_ratio:** Số lượng màu sắc duy nhất chia cho số sản phẩm duy nhất.
- **avg_value_per_item:** Giá trị trung bình của mỗi món hàng trong giỏ.
- **customer_return_tendency:** Tương tác giữa kinh nghiệm mua sắm và tỷ lệ trả hàng lịch sử ($\text{customer_return_rate} * \text{customer_order_number}$).

3. Tái cấu trúc thành Scikit-Learn Pipeline & Khắc phục rò rỉ dữ liệu

Nhằm chuẩn hóa mã nguồn để sẵn sàng cho môi trường deploy, toàn bộ quy trình xử lý dữ liệu và mô hình hóa được tích hợp vào một Pipeline thống nhất:

- **Custom Transformer ReturnFeatureExtractor:** Xây dựng một transformer tùy chỉnh kế thừa BaseEstimator và TransformerMixin. Transformer này thực hiện việc gộp nhóm các bảng quan hệ, tính toán các đặc trưng lịch sử lũy kế và các đặc trưng tương tác phi tuyến một cách tự động.
- **Xử lý Data Leakage:** Bằng cách áp dụng hàm `pd.merge_asof` kết hợp tham số `allow_exact_matches=False` trong ReturnFeatureExtractor, đảm bảo các đặc trưng lịch sử và Target Encoding của ngành hàng, vùng miền, phương thức thanh toán chỉ sử dụng thông tin của các sự kiện diễn ra trước ngày đặt hàng (`order_date`).
- **Đường ống (Pipeline) thống nhất:**
 1. **Feature Extraction:** Chuyển đổi dữ liệu thô thông qua ReturnFeatureExtractor.
 2. **Imputation & Encoding:** Thay thế các giá trị thiếu bằng SimpleImputer (giá trị median đối với đặc trưng số và 'unknown' đối với đặc trưng phân loại), sau đó mã hóa nhãn bằng OrdinalEncoder.
 3. **Classifier:** Huấn luyện mô hình XGBClassifier sử dụng tham số `scale_pos_weight=14.34` để bù đắp sự mất cân bằng lớp (tỷ lệ 93.5% : 6.5%).

4. Tích hợp MLflow Tracking

Quy trình huấn luyện đã được tích hợp đầy đủ công cụ MLflow Tracking để quản lý thử nghiệm:

- **Log thông số:** Ghi nhận toàn bộ các tham số của XGBoost (`max_depth`, `learning_rate`, `subsample`, v.v.) cùng tham số phân bổ trọng số `scale_pos_weight`.

- **Log chỉ số:** Lưu trữ kết quả AUC-ROC trên cả tập huấn luyện (Train) và tập kiểm thử (Test) cùng số vòng tối ưu tốt nhất (best_iteration).
- **Đóng gói Pipeline:** Lưu trữ toàn bộ Scikit-Learn Pipeline dưới dạng cloudpickle và đính kèm tệp định nghĩa return_pipeline.py để phục vụ việc tái tạo mô hình từ xa.

5. Xây dựng API FastAPI và Tối ưu hóa hiệu năng cực đại

Chúng tôi đã xây dựng Web API dự đoán bao gồm 3 endpoint chính: **/health**, **/predict** (dự đoán đơn lẻ) và **/predict/batch** (dự đoán hàng loạt).

Tối ưu hóa độ trễ (Khắc phục lỗi treo Swagger UI):

Vấn đề ban đầu: Trong phiên bản cũ, mỗi request dự đoán gửi đến /predict bắt buộc phải tính toán lại toàn bộ các chỉ số lũy kế động bằng cách gom nhóm dữ liệu lịch sử huấn luyện (>540,000 đơn hàng), dẫn đến thời gian phản hồi mất từ 30 giây đến vài phút và làm treo giao diện Swagger UI.

Giải pháp tối ưu:

- Dịch chuyển toàn bộ các phép gom nhóm và tính toán lịch sử lũy kế nặng nề vào hàm fit() để chạy duy nhất một lần khi huấn luyện.
- Lưu trữ các bảng dữ liệu lịch sử này thành thuộc tính bên trong instance của custom transformer và đóng gói trực tiếp cùng mô hình thông qua MLflow.
- Khi có request dự đoán mới gửi đến, mô hình trong hàm transform() chỉ thực hiện tra cứu nhị phân siêu tốc (pd.merge_asof) trên bộ nhớ RAM của máy chủ.

Kết quả: Độ trễ phản hồi của API /predict giảm từ hàng chục giây xuống dưới 10 mili-giây (phản hồi tức thì), giải quyết triệt để lỗi treo giao diện Swagger UI.

6. Thiết kế Feature Store trực tuyến với Redis Cloud

Khi triển khai mô hình lên môi trường sản xuất dưới dạng API thời gian thực, mô hình cần tra cứu tức thời các đặc trưng lịch sử của khách hàng và sản phẩm/ngành hàng để đưa ra dự báo ngay khi khách hàng checkout.

6.1. Bài toán đặt ra và Hạn chế của giải pháp In-Memory

Ở thiết kế ban đầu, toàn bộ dữ liệu tra cứu lịch sử được tính toán và lưu trực tiếp trong bộ nhớ RAM của máy chủ API. Cách tiếp cận này bộc lộ nhiều hạn chế lớn khi vận hành thực tế:

- **Tiêu tốn tài nguyên:** Máy chủ API phải duy trì một lượng lớn dữ liệu tra cứu (hơn 540,000 khách hàng và hàng ngàn sản phẩm) trên RAM.

- **Khó mở rộng (Scalability):** Khi cần chạy nhiều bản sao (replica) API song song để chia tải, việc đồng bộ bộ nhớ in-memory giữa các instance là bất khả thi.
- **Thời gian khởi động lâu:** Mỗi lần khởi động lại API, hệ thống phải tải toàn bộ dữ liệu thô từ database để tính toán lại các bảng tra cứu, làm tăng đáng kể thời gian downtime.

6.2. Giải pháp Redis Feature Store tập trung

Em đã tách biệt tầng lưu trữ đặc trưng bằng cách thiết kế một Online Feature Store sử dụng cơ sở dữ liệu Redis Cloud tập trung và hiệu năng cao.

Cấu trúc lưu trữ dữ liệu đặc trưng trên Redis:

- **Khách hàng:** Key cust:feat:<customer_id> => JSON string chứa thông tin lịch sử: số đơn hàng đã đặt, tổng chi tiêu, ngày đặt đơn gần nhất, số đơn đã trả, số review đã viết và tổng điểm đánh giá.
- **Sản phẩm:** Key prod:feat:<product_id> => JSON string chứa: số đơn hàng chứa sản phẩm này và số đơn bị trả lại.
- **Danh mục:** Key cat:feat:<dominant_category> => JSON string chứa tổng số đơn và số đơn bị trả lại của ngành hàng.
- **Khu vực:** Key reg:feat:<region> => JSON string chứa tổng số đơn và số đơn bị trả của vùng địa lý.
- **Phương thức thanh toán:** Key pay:feat:<payment_method> => JSON string chứa tổng số đơn và số đơn bị trả của phương thức đó.

6.3. Cơ chế truy vấn đặc trưng trực tuyến (Online Inference)

Kích hoạt động: Trong FastAPI, khi service khởi chạy (sử dụng lifespan), ứng dụng sẽ tự động kích hoạt tính năng Redis cho bộ trích xuất đặc trưng của mô hình bằng cách thiết lập:

`python`

`fe_step.redis_url = "redis://..."`

`fe_step.use_redis_ = True`

- **Truy vấn siêu tốc:** Trong phương thức transform(), thay vì thực hiện merge các bảng dữ liệu in-memory, mô hình sử dụng lệnh r.mget(...) của Redis để lấy đồng thời tất cả các khóa đặc trưng cần thiết dựa trên thông tin customer_id, product_id, dominant_category, region, và payment_method của đơn hàng mới.
- **Hệ quả vận hành:** Toàn bộ API trở thành stateless (không trạng thái), không cần nạp bất kỳ dữ liệu huấn luyện nào lên RAM khi khởi động. Thời gian đọc đặc trưng từ Redis Cloud chỉ mất dưới 10ms.